

Roshvyn

Setup & Deployment Guide From cloning the repository to a public API

Roshvyn Desktop (Mac & Windows) · Roshvyn API (Windows GPU PC)

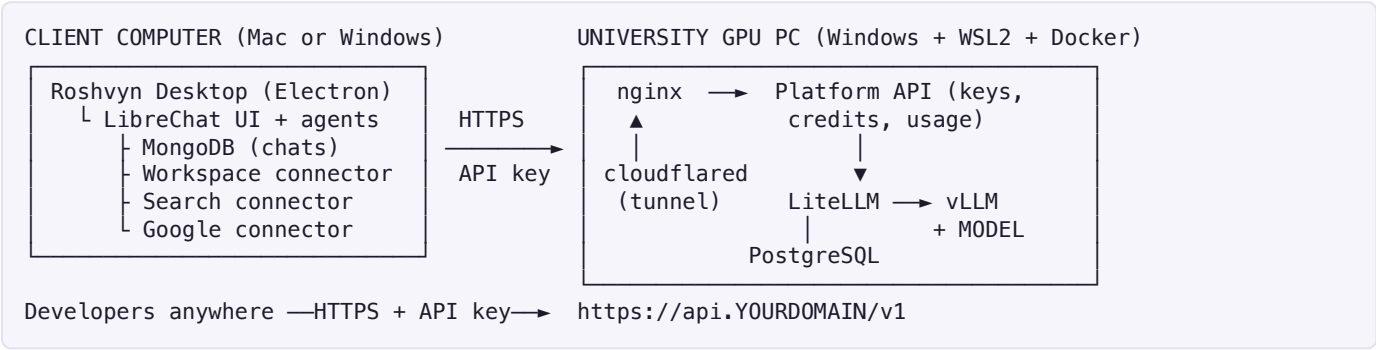
Version RV-DEMO-1 · 23 September 2026
Based on LibreChat v0.8.7 (commit 9e74cc0)

MAC **WINDOWS** **BOTH** tags show where each step runs

1. What you are building

Roshvyn is two connected products that share one repository:

- **Roshvyn API** — a developer platform: sign-up, administrator-granted credits, API keys, usage reporting, and an OpenAI-compatible model API. It runs on the university **GPU PC**.
- **Roshvyn Desktop** — a branded chat app (LibreChat inside an Electron window) that works on files, runs commands, searches the web and connects to Google Workspace. It runs on each **client computer** (your Mac or a Windows PC).



Golden rules

- The **model is downloaded only on the GPU PC**, never on the Mac.
- The Mac is for development and for the Desktop app; it may use a small stand-in model for testing.
- Secrets (passwords, API keys, tokens) live in files with owner-only permissions and are never committed to Git.

2. What you need before you start

Item	Where	Notes
Private GitHub repository	Both	Holds the code. Model files and databases are never stored in Git.
Git, Node.js 24.16, uv (Python), MongoDB Community	Client computer	Installed in section 3/5.
Windows 10/11 + NVIDIA GPU + WSL2 + Docker Desktop	GPU PC	16 GB VRAM target (RTX 4070 Ti SUPER).
About 60 GB free disk	GPU PC	Model ~10 GB, container images ~15 GB, database + backups.
A domain name + access to its DNS	For public access	Used as <code>api.YOURDOMAIN</code> and <code>console.YOURDOMAIN</code> .
Cloudflare account (free)	For public access	Runs the named tunnel; no router ports are opened.
Google Cloud OAuth client (optional)	Client computer	Only for Gmail/Drive/Calendar/Docs/Sheets features.

3. **MAC** Set up a client computer (Mac)

3.1 Install the basic tools

```
# Xcode command line tools (git, compilers)
xcode-select --install
# uv = Python version and environment manager
curl -LsSf https://astral.sh/uv/install.sh | sh
# MongoDB Community Server (database for chats), via Homebrew
brew tap mongodb/brew && brew install mongodb-community
```

3.2 Clone the repository

```
cd ~/Desktop
git clone https://github.com/OWNER/roshvyn.git roshvyn
cd roshvyn
git checkout demo-v0.1.0          # the tested release tag
```

3.3 Install Node.js 24.16.0 inside the project

The project uses its own Node so your system Node stays untouched.

```
mkdir -p .local-dev/tools && cd .local-dev/tools
curl -L0 https://nodejs.org/dist/v24.16.0/node-v24.16.0-darwin-x64.tar.gz
curl -L0 https://nodejs.org/dist/v24.16.0/SHASUMS256.txt
grep node-v24.16.0-darwin-x64.tar.gz SHASUMS256.txt | shasum -a 256 -c  # must print OK
tar xzf node-v24.16.0-darwin-x64.tar.gz && rm node-v24.16.0-darwin-x64.tar.gz
export PATH="$PWD/node-v24.16.0-darwin-x64/bin:$PATH" && node -v
cd ~/Desktop/roshvyn
```

On an Apple Silicon Mac use `node-v24.16.0-darwin-arm64.tar.gz` instead.

3.4 Build LibreChat (the chat interface)

```
npm ci          # installs dependencies (several minutes)
npm run frontend # builds the web interface
```

3.5 Create the Python environments

```
for d in roshvyn/platform-api roshvyn/infra/litellm roshvyn/infra/mock-inference \
        roshvyn/connectors/workspace roshvyn/connectors/search roshvyn/connectors/google
roshvyn/tests; do
  (cd $d && uv sync --frozen)
done
# browser used for page checks and tests
PLAYWRIGHT_BROWSERS_PATH=$PWD/.local-dev/playwright-browsers \
uv run --project roshvyn/tests playwright install chromium
```

3.6 Install the Electron shell

```
cd roshvyn/desktop && npm install && node node_modules/electron/install.js && cd ../..
```

4. MAC Optional: run the platform locally for development

You only need this if you want the whole API platform on the Mac (useful before the GPU PC is ready). If your GPU PC is already serving, skip to section 5 and point the Desktop app at it.

4.1 Local database and services

The development stack uses a portable PostgreSQL inside the project folder, then starts everything:

```
roshvyn/scripts/mac/dev-platform.sh start mock          # fake model: instant, replies labelled  
MOCK  
roshvyn/scripts/mac/dev-platform.sh start dev-local-4b  # small real model on the CPU (slow)  
roshvyn/scripts/mac/dev-platform.sh status  
roshvyn/scripts/mac/dev-platform.sh stop
```

What starts: PostgreSQL (5433), the model upstream, LiteLLM gateway (4000), the platform API (8200) and the portal (5173). Everything listens on 127.0.0.1, meaning this Mac only.

4.2 Create the first administrator and the model entry

```
cd roshvyn/platform-api  
uv run python -m app.cli bootstrap-admin --email admin@example.com  # asks for a password  
uv run python -m app.cli create-gateway-key                        # prints a key  
# paste that key into roshvyn/platform-api/.env as ROSHVYN_PLAYGROUND_GATEWAY_KEY, then restart  
the API
```

Open <http://127.0.0.1:5173>, log in as the admin, and you can create customers, grant credits and issue API keys.

5. BOTH Run Roshvyn Desktop

5.1 Give the app an API key

In the portal (your own server, or the GPU PC once deployed): **API keys** → **Create key**. Copy the key; it is shown once.

5.2 Start it MAC

```
# store the key for the app (owner-only file)  
mkdir -p .local-dev/client/secrets && chmod 700 .local-dev/client/secrets  
printf '%s' 'sk-YOUR-KEY' > .local-dev/client/secrets/roshvyn-api-key  
chmod 600 .local-dev/client/secrets/roshvyn-api-key  
  
# point it at your server (leave out to use a local platform)  
export ROSHVYN_API_BASE=https://api.YOURDOMAIN/v1  
roshvyn/scripts/mac/start-desktop.sh
```

5.3 Start it WINDOWS

```
powershell -ExecutionPolicy Bypass -File roshvyn\scripts\windows\Start-Desktop.ps1
```

Prerequisites on Windows: Git, Node 24.16.0, uv, MongoDB Community (`mongod.exe` on PATH), the same `npm ci / npm run frontend / uv sync` steps as sections 3.4–3.6. Put the API key in `%APPDATA%\Roshvyn\secrets\roshvyn-api-key`, or paste it in the app's first-run window.

5.4 Create the local chat account (first run only)

The Desktop chat keeps its own local account, separate from the portal account.

```
# Mac (from the repository root, with the services running)
MONGO_URI=mongodb://127.0.0.1:27028/RoshvynDesktop \
CONFIG_PATH=$PWD/.local-dev/client/librechat.yaml \
.local-dev/tools/node-v24.16.0-darwin-x64/bin/node config/create-user.js you@example.com "Your
Name" you --email-verified=true
```

5.5 First use

1. The top bar shows your **workspace folder** and **Model API: online**. Use **Change folder** to pick which folder the assistant may touch.
2. Log in with the local account, pick the **Roshvyn Coder** agent, and ask for something, for example: *“Create a folder demo-notes with a README.md explaining Roshvyn, then list it.”*
3. Normal build and test commands (npm, python, pytest...) run automatically. Anything else shows an **approval window** with the exact command and Deny / Approve once.

Safety: the assistant can only touch the selected workspace folder. Approved commands run with your user permissions, so read them before approving. This is not a full sandbox.

6. WINDOWS Deploy the server on the GPU PC

This is where the real model lives. Every block says whether it is **PowerShell** (Windows) or **WSL** (the Ubuntu terminal).

6.1 Check the machine — PowerShell

```
powershell -ExecutionPolicy Bypass -File roshvyn\scripts\windows\Preflight.ps1 -GpuContainerTest
```

It checks Windows, RAM, disk, the NVIDIA driver, WSL2, Docker and port 8080, and writes `preflight-result.json`. Fix anything marked CHECK:

- **NVIDIA driver:** install the latest Windows driver. Never install Linux display drivers inside WSL — WSL uses the Windows driver.
- **WSL2:** `wsl --install -d Ubuntu-24.04` in an admin PowerShell, then reboot.
- **Docker Desktop:** install, then enable *Use the WSL 2 based engine* and WSL integration for Ubuntu.

6.2 Get the code — WSL

```
sudo apt-get update && sudo apt-get install -y git curl openssl python3
curl -LsSf https://astral.sh/uv/install.sh | sh
cd ~ && git clone https://github.com/OWNER/roshvyn.git roshvyn
cd ~/roshvyn && git checkout demo-v0.1.0
```

Keep it in `~/roshvyn` (the Linux filesystem), not under `/mnt/c`, for speed.

6.3 Create the server secrets — WSL

```
roshvyn/scripts/server/init-secrets.sh
nano .server-secrets/server.env      # set OWNER_DOMAIN and ROSHVYN_MODEL_DIR
```

6.4 Check the tool-call parser (do not guess) — WSL

```
docker run --rm vllm/vllm-openai:v0.29.0 --help | grep -A6 -- '--tool-call-parser'
```

Put the correct value into `ROSHVYN_TOOL_PARSER` in `server.env` (`hermes` for the Qwen3-4B-Instruct baseline; the Qwen3.5 model card uses `qwen3_coder`).

6.5 Download the model — WSL

```
roshvyn/scripts/server/download-model.sh
```

Downloads the pinned revision to your SSD (outside Git) and writes `SHA256SUMS` and a provenance file. Takes a while: the model is about 8–10 GB.

6.6 Build the portal and start everything — WSL

```
cd ~/roshvyn/roshvyn/portal && npm ci && npm run build && cd ~/roshvyn
roshvyn/scripts/server/start.sh      # local only; no public access yet
roshvyn/scripts/server/health.sh
```

`start.sh` builds the images, starts PostgreSQL and vLLM (the model load takes minutes), waits for each service to become healthy, then LiteLLM, the platform API and nginx on `127.0.0.1:8080`. It prints which model is deployed and the GPU memory in use.

6.7 First administrator, model catalog, playground key — WSL

```
roshvyn/scripts/server/bootstrap-admin.sh admin@YOURDOMAIN

docker exec roshvyn-platform-api-1 python -m app.cli seed-model --alias roshvyn-coder \
  --identity "Qwen/Qwen3-4B-Instruct-2507 BF16, vLLM v0.29.0, RTX 4070 Ti SUPER" --context 4096
--max-output 1024

docker exec roshvyn-platform-api-1 python -m app.cli create-gateway-key
nano .server-secrets/platform.env # paste as ROSHVYN_PLAYGROUND_GATEWAY_KEY, set
ROSHVYN_PUBLIC_API_BASE
docker compose -f roshvyn/infra/server/compose.yaml --env-file .server-secrets/server.env up -d
platform-api
```

6.8 Test locally before going public — WSL

```
curl -s -H "Host: api.YOURDOMAIN" http://127.0.0.1:8080/v1/models # expect 401 without a
key
```

Then temporarily add `127.0.0.1 console.YOURDOMAIN api.YOURDOMAIN` to

`C:\Windows\System32\drivers\etc\hosts`, open the console in a browser, register a customer, grant credits, create a key, and run:

```
ROSHVYN_BASE_URL=http://api.YOURDOMAIN:8080/v1 ROSHVYN_API_KEY=sk-... \
uv run --project roshvyn/tests python roshvyn/scripts/test-public-api.py
```

Remove the hosts lines afterwards.

7. Put it on the internet (Cloudflare Tunnel)

A tunnel makes an **outgoing** connection from the PC to Cloudflare, so you do not open any port on the university router or firewall.

1. Add your domain to Cloudflare (change the nameservers at your registrar). Do not delete existing DNS records you rely on.
2. Cloudflare dashboard → **Zero Trust** → **Networks** → **Tunnels** → **Create a tunnel** (type: cloudflared). Copy the tunnel **token**.
3. **WSL**: paste the token into `.server-secrets/cloudflared_token`.
4. In the tunnel's **Public hostnames**, add two entries, both pointing to `http://nginx:8080`:
 - `api.YOURDOMAIN` — the model API for developers
 - `console.YOURDOMAIN` — the web portal
5. **WSL**: `roshvyn/scripts/server/start.sh --with-tunnel`

Do not put Cloudflare Access (the browser login page) in front of `api.` — SDK clients cannot log in through a web page, and every request would fail.

7.1 Test from a different internet connection

```
export ROSHVYN_BASE_URL=https://api.YOURDOMAIN/v1
export ROSHVYN_API_KEY=sk-...
uv run --project roshvyn/tests python roshvyn/scripts/test-public-api.py
# revoke the key in the portal, then confirm it stops working:
uv run --project roshvyn/tests python roshvyn/scripts/test-public-api.py --expect-revoked
```

Also confirm these return “not found”: `https://api.YOURDOMAIN/key/generate` and `https://console.YOURDOMAIN/v1/models`.

7.2 What a developer now uses

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="roshvyn-coder",
                  base_url="https://api.YOURDOMAIN/v1",
                  api_key="sk-...", use_responses_api=False)
print(llm.invoke("Write a Python function that validates a task title.").content)
```

The same key works with the OpenAI SDK and plain curl. The base URL always ends at `/v1` — never add `/chat/completions` to it.

7.3 Testing without a domain (temporary)

To practise on a Mac or before the domain is ready, a quick tunnel gives a random temporary address:

```
# terminal 1: publish ONLY the model API (blocks the portal and admin routes)
uv run --project roshvyn/platform-api python roshvyn/infra/edge-proxy.py --host 127.0.0.1
# terminal 2: temporary public HTTPS address
cloudflared tunnel --url http://127.0.0.1:8300
```

Use the printed `https://...trycloudflare.com/v1` as the base URL. Press Ctrl+C to take it offline. This is for learning only: the address changes each time and your laptop is doing the serving.

8. Daily operations WINDOWS GPU PC

Task	Command (WSL)
Start everything	<code>roshvyn/scripts/server/start.sh --with-tunnel</code>
Check health	<code>roshvyn/scripts/server/health.sh</code>
Stop (keeps all data)	<code>roshvyn/scripts/server/stop.sh</code>
Backup databases	<code>roshvyn/scripts/server/backup.sh</code> then encrypt the folder
Update to a new release	<code>git fetch && git checkout <tag></code> then <code>start.sh</code>
View logs	<code>docker compose -f roshvyn/infra/server/compose.yaml logs -f platform-api</code>

Never run `docker compose down -v` — the `-v` deletes the database volume with all accounts, credits and usage history.

Keep it available: set Windows sleep to “Never” while hosting, and remember that after a reboot someone must log in so Docker Desktop starts, then run `start.sh --with-tunnel`. This is not an unattended server.

9. Troubleshooting

Symptom	Cause and fix
Portal login fails	The portal and the Desktop app have separate accounts. Use the portal account. After 5 wrong passwords an email is blocked for 15 minutes.
402 insufficient credits	The account has no credits. An administrator grants them in Users & credits .
401 invalid API key	Key revoked, expired, or copied incompletely. Create a new one.
503 billing database unavailable	PostgreSQL is down. The system refuses work rather than running untracked requests. Check <code>health.sh</code> .
429	Too many requests at once for this demo capacity. Retry shortly.
vLLM will not start / out of memory	Lower <code>R0SHVYN_MAX_MODEL_LEN</code> (e.g. 4096) or <code>R0SHVYN_GPU_MEM_UTIL</code> in <code>server.env</code> , then <code>start.sh</code> .
Model gives text instead of using a tool	Wrong <code>R0SHVYN_T00L_PARSER</code> for the model. Re-check section 6.4.
Tunnel shows “down” in Cloudflare	University network may block it. Report it; do not attempt to bypass network controls.
Desktop: “Model API: unreachable”	Wrong <code>R0SHVYN_API_BASE</code> , server stopped, or no internet. Test with <code>curl \$R0SHVYN_API_BASE/models</code> .
Assistant says a command needs approval but no window appears	The Desktop app is not running (the approval window lives in it). Start it, or use an ordinary <code>build/test</code> command.
Port already in use	The scripts refuse to touch processes they do not own. Stop the other program or change the port in <code>config/demo.env</code> .

10. Reference

Ports (all local unless tunnelled)

Port	Service	Where
5173	Portal (development)	Client / dev Mac
8200	Platform API	Dev Mac / GPU PC (internal)
8300	Edge proxy (public API only)	Mac, for tunnelling practice
4000	LiteLLM gateway	Internal only
8000	vLLM + model	GPU PC, internal only
8080	nginx (behind the tunnel)	GPU PC
3090	LibreChat (Desktop)	Client computer
27028	MongoDB (chats)	Client computer
8888	SearXNG (web search)	Client computer
3099	Approval bridge	Client computer

Important files

Path	Contents
<code>.server-secrets/</code> <code>GPU PC</code>	Database passwords, gateway key, tunnel token, <code>server.env</code> . Never committed.
<code>.local-dev/</code> <code>MAC</code>	Development database, tools, logs and secrets. Never committed.
<code>roshvyn/docs/</code>	STATUS, ARCHITECTURE, API-CONTRACT, CREDITS-AND-RECOVERY, GOOGLE-SETUP, MODEL-EVALUATION, KNOWN-LIMITATIONS, DEMO-SCRIPT.
<code>roshvyn/scripts/</code>	<code>mac/</code> , <code>windows/</code> and <code>server/</code> start, stop, health and deployment scripts.

Security summary

- Only two public routes exist: `GET /v1/models` and `POST /v1/chat/completions`. The gateway, model, databases and desktop tools are never routed publicly.
- Credits are reserved before generation and settled on actual usage; the ledger cannot be edited or deleted.
- API keys are stored as one-way fingerprints — a lost key cannot be recovered, only replaced.
- The Desktop assistant is limited to the chosen workspace folder; other commands need your approval.
- Web pages fetched by the assistant are treated as untrusted data, and private network addresses are blocked.

Read `roshvyn/docs/KNOWN-LIMITATIONS.md` for what is not yet implemented or tested, including Windows testing, the Google connector and GPU acceptance tests.